Episcaf: project log

Brandon Neff John Altin
TGen

June 21, 2026

Living record. Read top to bottom to get up to speed. Figures regenerated from `results/`; see `docs/WORKFLOW.md`

This is the running record of the episcaf project: scaffolding antibody epitopes defined by 3D geometry into folded, assay-ready designs (RFdiffusion3 → ProteinMPNN → AlphaFold3). It is meant to be read in order, since each section is one iteration: what we tried, what we found, and what it changed for the next step. It is not a formal manuscript. Every figure is regenerated by a named script (recorded in `manuscript/figures/FIGURES.md`) and every number traces to `results/`; unconfirmed values are flagged `[verify]`.

1 Motivation: scaffolding B-cell epitopes

1.1 Why scaffold epitopes

This project is about *B-cell epitopes*: the regions of an antigen that an antibody recognizes. Antibodies recognize these regions through their three-dimensional geometry, and many of the most interesting B-cell epitopes are discontinuous in sequence but contiguous in space, meaning the contacted residues are scattered along the antigen sequence yet fold together into a single surface patch. Linear methods cannot represent this. High-throughput serology platforms such as PepSeq tile an antigen into short, overlapping linear peptides, which works well for continuous, sequence-local epitopes. A linear peptide, however, cannot reconstitute a surface assembled from sequence-distant residues, and it discards the native conformation the antibody evolved to recognize.

The central question of this project is whether *scaffolding* an epitope produces a stronger, cleaner binding signal in the PepSeq assay than linear tiling does. By embedding the epitope residues in a folded protein, we can hold them in the conformation the antibody actually sees, and we can present distinct epitopes as separable, individually testable designs. If that conformational presentation matters, scaffolded designs should bind their cognate antibodies more reliably than the corresponding linear peptides. The longer-term goal is to extend PepSeq-style high-throughput assays beyond linear peptides to conformational, discontinuous epitopes.

1.2 The approach

We extract the epitope, meaning the residues an antibody contacts, from an antibody:antigen crystal structure. We embed those residues into a *de novo* protein scaffold with RFdiffusion, assign a sequence with ProteinMPNN, and then ask AlphaFold3 whether the design actually folds with the epitope held in its native geometry. Designs are kept or rejected on AlphaFold3 confidence and geometry-preservation metrics (Section 2). The full production pipeline, including the Nextflow orchestration, the RFdiffusion runs, and the AlphaFold3 evaluation, originates in Lawson Woods'

`episcaf-experiments` repository. Reproducing and then extending it is the starting point of this work.

1.3 Patches and islands

A single antibody:antigen complex can yield more than one epitope *patch*, labeled `0P`, `1P`, and so on, where each patch is a distinct cluster of antibody-contacting residues drawn from the same structure. A patch is in turn made of one or more *islands*, the spatially contiguous runs of epitope residues in three-dimensional space. This distinction matters because it predicts difficulty. Single-island patches can be enforced locally and tend to scaffold well, whereas multi-island patches require preserving the relative position and orientation of spatially separated regions, which is much harder. A concrete failure makes the point: PDB `2XQB`, patch `0P`, is a 56-residue epitope split across two separated islands, and across 2,624 RFdiffusion trials zero designs passed the downstream filters. Because this 0% rate holds over thousands of attempts, it looks like a real computational failure, not undersampling. The cause appears procedural: contigs (the constraints RFdiffusion is given) enforce epitope residue identity and ordering, but they do not strongly constrain the distances and orientations *between* islands, so the model can satisfy the contig and still lose the geometry an antibody needs.

1.4 Two design pools: DP3 and DP4

The project runs in two settings that differ in how much we know about the antibody.

- **DP3 (known antibody).** When the antigen has a solved antibody complex, we know the exact epitope and we have a ground truth for "good": the design must hold the epitope in place and must not place scaffold mass where the antibody has to bind. This is the setting where we have already run designs through the assay and seen some success, and it is where the question of scaffolded-versus-linear signal is most directly testable.
- **DP4 (tiled 12-mers, no known antibody).** For most antigens we have no solved antibody, so we do not know where the epitope is. Here we instead tile the antigen sequence into overlapping 12-residue windows, stepping by two residues at a time, and scaffold each window. The aim is to cover the antigen densely enough to catch whatever the real epitopes turn out to be.

1.5 Why a cylinder: the antibody-clash filter without an antibody

One of the strictest filters in the DP3 setting concerns antibody accessibility. We superimpose the design's epitope region onto the native antigen, place the real antibody where it sits in the crystal complex, and count scaffold residues that clash into the antibody. Any design with a clash count above zero is rejected, because scaffold mass in the antibody's path would block the very binding event we are trying to recreate. This filter is only available for DP3, where a real antibody exists. For DP4 there is no antibody to clash against, so we cannot apply it directly. The native-antigen-aware cylinder surrogate (Section 7) is our attempt to recover this filter without an antibody: it approximates the volume an approaching antibody would occupy and asks whether scaffold mass intrudes into it. A single, config-driven composite scorer ranks designs in both pools, differing only in which accessibility term it uses.

1.6 How the rest of this log reads

The sections that follow are in the order the work happened. We start by reproducing Lawson's workflow and confirming our metrics match his (Section 2), cover how we tile antigens into assay constructs (Section 3), and compare the RFdiffusion1 and RFdiffusion3 design sets directly (Section 4). We then turn to the filtering, where the original global metrics bias selection toward rigid, well-ordered folds, and show how decomposing them widens the designable space (Section 5). From there we build a composite score that selects the best designs per epitope (Section 6), and finally extend everything to DP4 using the native-aware cylinder surrogate (Section 7). Each step motivates the next.

2 The pipeline, and reproducing Lawson's workflow

2.1 Pipeline

Each contig is carried through three stages. RFdiffusion3 generates 8 scaffold backbones holding the epitope segment fixed at its contig position (one GPU task per contig, `n_batches=1`). Each backbone is stripped to its $C\alpha$/backbone trace and re-sequenced by ProteinMPNN, which is told to hold the epitope residues *fixed* — so it designs only the scaffold — and draws 8 sequences per backbone (temperature 0.1); the epitope's fixed positions are read back from the contig, since an island sits at residues $N{+}1 \ldots N{+}\ell$ of an N-flank contig. AlphaFold3 then predicts each sequence single-sequence (`--norun_data_pipeline`, one diffusion sample, seed 1), giving the $8 \times 8 = 64$ designs per contig that the four filters score against the designed structure. Generation code lives in `episcaf_pipeline/` and `scripts/`, and the metrics, scoring, and visualization code in `episcaf_analysis/`.

Benchmarking. Rough per-step throughput, to size cluster wall-time requests: RFdiffusion3 produces a contig's 8 backbones in under 10 min on one A100; ProteinMPNN sequences ~ 215 backbones/hour (≈ 17 s for a backbone's 8 sequences); AlphaFold3 single-sequence predictions are fast but unmeasured on our V100 nodes [verify]. A run's wall-time then follows by multiplication — e.g. a 300-backbone ProteinMPNN batch is ≈ 1.4 h, for which we request 8 h to leave margin.

2.2 Reproducing Lawson's baseline

Before trusting any new number we re-implement Lawson's metrics and check them against his ground truth. `compute_metrics.py --validate` reads `dp2.parquet` (Lawson's stored per-design values), recomputes `overall_rmsd` and `epitope_chunk_rmsd` from his ProteinMPNN PDBs and AlphaFold3 CIFs, and diffs them against the stored values. Agreement on this diff is the precondition for everything downstream: it confirms that our implementation of the filters matches the workflow we are extending, so that later differences reflect the design protocol rather than a scoring bug.

2.3 Four-filter ground truth (DP3 set)

A design passes when all four conditions hold: overall backbone RMSD ≤ 2 Å, epitope-chunk RMSD ≤ 1 Å, mean predicted aligned error < 5, and zero antibody residues within 4 Å of non-epitope scaffold residues (`af3_n_clash_res`). Clash detection Kabsch-aligns the AlphaFold3 epitope onto the true epitope, then checks non-epitope scaffold heavy atoms against antibody atoms. The antibody coordinates always come from the AbDb crystal structure, not from AlphaFold3.

We summarize a design protocol by its empirical pass rate. If a protocol produces N designs s_i and $f(s_i) = 1$ when design i clears all four filters (and 0 otherwise),

$$\hat{P}_{\text{pass}} \;=\; \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[f(s_i) = 1]\,. \tag{1}$$

This is just the fraction of designs that pass, and it is the quantity we compare between protocols and report per epitope.

2.4 Decomposed metrics

The two continuous filters, overall RMSD and mean PAE, treat the epitope and the scaffold as a single unit. We split each into epitope- and scaffold-specific terms (epitope RMSD, scaffold RMSD, epitope PAE, scaffold PAE), retaining the zero-clash hard filter throughout. Decomposition lets selection reward a well-placed epitope on a flexible scaffold (or the reverse) instead of demanding global rigidity, and it lets us ask whether the optimal rigidity profile is epitope-dependent rather than universal (Section 5). Selection over the four metrics is organized as a 2×2 of strict and relaxed thresholds on (epitope, scaffold), with a flexible/flexible arm as a negative control.

2.5 Native-antigen-aware cylinder surrogate (12-mer set)

With no antibody available, we approximate the volume an antibody would occupy with a cylinder fitted to the epitope. The epitope Cα coordinates $\{x_j\}_{j=1}^{m}$ define a plane: the centroid is $c = \frac{1}{m}\sum_j x_j$, and the plane normal $\hat{n}$ is the smallest-variance singular vector of the centered coordinates $\{x_j - c\}$ (the antibody approach direction), oriented to point away from the antigen body. The cylinder base is $b = c + d_0\,\hat{n}$ with offset $d_0 = -4\,\text{Å}$. A scaffold Cα at position p lies inside the cylinder when, writing $v = p - b$, its axial and radial coordinates satisfy

$$0 \;\le\; v \cdot \hat{n} \;\le\; H \qquad \text{and} \qquad \big\| \, v - (v \cdot \hat{n})\,\hat{n} \, \big\| \;\le\; R, \tag{2}$$

with height $H = 40\,\text{Å}$ and radius $R = 16\,\text{Å}$. The plain count is the number of non-epitope scaffold Cα satisfying (2); the native-aware count additionally drops any such atom within 1 Å of a native-antigen heavy atom, i.e. sitting where the native antigen already sits, which a real antibody tolerates. The geometry core is `episcaf_analysis/native_cylinder_core.py` (self-tested).

2.6 Composite scorer

One config-driven scorer (`episcaf_analysis/score.py`) ranks designs in four steps: (i) **gate** on epitope-chunk RMSD; (ii) transform each metric to a higher-is-better score within a scope (pooled, or per antigen); (iii) **weighted sum**; and (iv) keep the **top-k per epitope**. Concretely, each design d receives a score

$$S(d) \;=\; \sum_{m} w_m\, \varphi_m\big(x_m(d)\big), \tag{3}$$

a weighted sum over metrics m of a per-metric activation φ_m that maps the raw value x_m to a higher-is-better score. With φ a percentile the score is rank-based and depends on the cohort; with a sigmoid,

$$\varphi(x) \;=\; \frac{1}{1 + \exp\big(s\,k\,(x - x_0)\big)}, \qquad s = +1 \text{ if lower is better (else } -1), \tag{4}$$

it is an absolute, saturating score with a tunable midpoint x_0 and steepness k. In this form the scorer is literally a single-layer perceptron with hand-set weights (feed-forward, no backpropagation): the activations are the per-metric nonlinearities, the w_m are the weights, and "fitting" it (Section 9) means learning the w_m (and the x_0, k) from data. The dials live in `episcaf_analysis/presets.py`:

metric	weight
epitope-chunk RMSD	0.35
epitope PAE	0.25
accessibility	0.25
overall RMSD	0.15

The weights sum to one (renormalized over whichever metrics a given table actually contains). The only term that changes between the two regimes is accessibility: it is `cylinder_native_aware` for the no-antibody 12-mer set and `cylinder_ca_clashes` for the DP3 set, using the same cylinder geometry, with the native-aware carve applied only where there is no real antibody to clash against. The 12-mer preset transforms metrics per antigen and keeps the top 5 designs per epitope, while the DP3 preset transforms pooled and keeps the top 15. The weights are a hand-set starting point; Section 9 describes fitting them to the DP3 `is_pass` label.

2.7 Per-island scaffolding of dual-island epitopes

A two-island epitope binds its antibody through two spatially separate patches, and the native-antigen geometry alone cannot say how the binding energy is divided between them. Scaffolding each island *on its own* turns that question into an experiment: presenting island A in isolation, island B in isolation, and the two together (the native arrangement) lets the PepSeq assay deconvolve the contributions — whether the antibody binds through island A, through island B, or only through both, and whether different two-island epitopes distribute their binding differently across their islands. This complements the island-*count* comparison of Section 8, which asks whether two islands are harder to scaffold than one; here we instead build the reagents to dissect a single two-island epitope.

We define an *island* as one fixed-coordinate `A`-segment of the RFdiffusion3 contig string. For example the dp2 contig `15-15/A1-16/57-57/A81-81/15-15` has two islands, `A1-16` (16 residues) and `A81-81` (1 residue), separated by generated scaffold of the given lengths. A *dual-island epitope* is a ledger id with `epitope_chunks` = 2, i.e. exactly two such segments; the DP3 set contains 46 of them. We take *island size* to be the `A`-segment span in residues (not the count of antibody-contact residues within it).

Following John's plan, each dual-island epitope contributes its two islands as separate design targets, and we keep the top 5 designs per island under the composite scorer. The island lengths vary widely — from a single residue to 51 — and Table 1 lists both islands of all 46 epitopes so the run is auditable epitope by epitope. One case needs a rule: an island of a single residue cannot be presented on its own. Rather than drop the whole epitope, we simply *skip the size-*1 *island and still scaffold its partner*. Five islands are skipped on this ground, one each in `2h32_0P`, `3q1s_0P`, `6o9i_0P`, `7a3t_0P`, and `7tzh_0P` (marked in Table 1); every epitope still contributes at least one island. Of the 92 islands across the 46 epitopes, 87 are scaffolded, giving 87 individual-island targets and, at five designs each, 435 final designs. The per-island target list (both spans, sizes, and the scaffold/skip flag) and Table 1 itself are generated by `episcaf_analysis/dual_island_targets.py` into `results/dual_island_targets.csv` and `manuscript/tables/`, the single reproducible source for these counts.

Each island is scaffolded into a fixed 103-residue construct — the maximum peptide length the PepSeq assay synthesizes. We call the stretches of generated scaffold lying N-terminal and C-terminal to a fixed island its *flanks*; a single-island contig is therefore ⟨N-flank⟩/ `A`*a*-*b* /⟨C-flank⟩, and the scaffold budget the flanks share is $103 - \ell$ residues for an island of length ℓ. We note one unexplained convention difference: Lawson's original dual-island contigs were all 104 residues, not 103. The change to 103 appears to have been made at some point without a recorded reason; we adopt 103 as the assay ceiling and expect a single terminal residue, averaged over sequence space, to be immaterial, but flag it here because its origin is unknown.

Contig sampling and run size. A *contig* pairs the fixed island(s) with one concrete scaffold-length layout, and each contig is sampled at 8 RFdiffusion3 backbones × 8 ProteinMPNN sequences = 64 designs (the per-contig depth throughout this project). The run size is thus set by how many contigs we generate. We record how Lawson reached his 151 232-design DP3 set, so the scale here is never mysterious: for each epitope he held the two islands fixed and swept the scaffold layout, keeping the two flanks balanced ($|\mathrm{N} - \mathrm{C}| \leq 1$) and stepping the flank length over every integer from 1 to about 20, which makes the *inter-island gap* sweep from ∼46 down to a floor near 8 (the islands are never allowed to touch). This is a deterministic enumeration, not random sampling: it yields a median of ∼41 contigs per epitope, so 59 epitopes × 41 ≈ 2363 contigs and 2363 × 64 = 151 232 designs. The quantity his contigs actually vary is the spacing between the two islands.

Our single-island targets have no inter-island gap, so that lever does not exist; the only scaffold freedom is the *flank asymmetry* — where along the 103-mer the island sits, i.e. the split of the $103-\ell$ budget into N- and C-flank. We therefore generate V contigs per island by *randomly* sampling V distinct N-flank lengths over $[1,\ 103 - \ell - 1]$ (under a fixed seed, so the ledger is reproducible). This is an explicitly different diversity axis from Lawson's — scaffold position and topology, not inter-island geometry — which we name here rather than leave implicit. With V variants the run is

$$\underbrace{87}_{\text{islands}} \times V \times \underbrace{8}_{\text{RFD3}} \times \underbrace{8}_{\text{MPNN}} \;=\; 5568\,V \text{ designs.} \tag{5}$$

We use $V = 20$, giving 1740 contigs and 111 360 designs — the order of the previous DP3 set ($V \approx 27$ would reach its full ∼150 000). V is the `--variants` dial of `episcaf_pipeline/build_dual_island_designs.py`. Each RFdiffusion3 task emits 8 backbones (`n_batches=1`), so the 8-per-contig depth is one task, not eight: the RFD3 step is 1740 array tasks (one per contig), each yielding 8 backbones that ProteinMPNN then re-sequences 8 ways.

Table 1: Island lengths of the 46 dual-island DP3 epitopes, the input to the per-island design run (Section 2.7). Each island is one contig `A`-segment; the value in parentheses is its length in residues. An island marked † is a single residue and is skipped (it cannot be presented alone); the epitope's other island is still scaffolded. Of the 92 islands, 87 are scaffolded (size ≥ 2) and 5 skipped.

epitope	island 1	island 2	designed
`2h32_0P`	`A1--16` (16)	`A81--81` (1)†	larger only
`3q1s_0P`	`A14--30` (17)	`A107--107` (1)†	larger only
`6o9i_0P`	`A6--23` (18)	`A54--54` (1)†	larger only
`7a3t_0P`	`A150--150` (1)†	`A298--300` (3)	larger only

Table 1 (continued)

epitope	island 1	island 2	designed
7tzh_0P	A109--134 (26)	A148--148 (1)†	larger only
5eu7_1P	A12--13 (2)	A91--126 (36)	both
5fhx_1P	A14--15 (2)	A62--74 (13)	both
6ztr_0P	A16--17 (2)	A43--63 (21)	both
4u6v_0P	A152--179 (28)	A242--245 (4)	both
5j13_0P	A36--49 (14)	A103--106 (4)	both
6okm_0P	A39--42 (4)	A54--69 (16)	both
6qb6_0P	A40--43 (4)	A143--157 (15)	both
7uxl_1P	A56--80 (25)	A122--125 (4)	both
2xqb_0P	A19--23 (5)	A35--85 (51)	both
4kv5_3P	A28--32 (5)	A90--101 (12)	both
4nnp_0P	A198--210 (13)	A230--234 (5)	both
6ppg_1P	A55--69 (15)	A105--109 (5)	both
5gjs_0P	A12--17 (6)	A29--45 (17)	both
5o4g_0P	A129--134 (6)	A150--169 (20)	both
7so5_0P	A15--27 (13)	A56--61 (6)	both
8cz8_1P	A107--116 (10)	A139--144 (6)	both
2qqn_0P	A21--29 (9)	A45--51 (7)	both
6ol6_1P	A52--59 (8)	A100--106 (7)	both
6wgl_0P	A38--47 (10)	A66--72 (7)	both
8oxv_0P	A242--270 (29)	A627--633 (7)	both
3ma9_0P	A19--64 (46)	A186--193 (8)	both
5l6y_0P	A10--17 (8)	A82--93 (12)	both
6cyf_0P	A36--43 (8)	A104--111 (8)	both
7ox3_0P	A6--13 (8)	A63--75 (13)	both
8wsq_0P	A350--371 (22)	A387--394 (8)	both
3ux9_1P	A18--32 (15)	A115--123 (9)	both
4xwo_5P	A197--218 (22)	A249--257 (9)	both
6wmw_1P	A4--25 (22)	A69--77 (9)	both
8db4_0P	A6--18 (13)	A54--62 (9)	both
7kql_0P	A28--49 (22)	A89--98 (10)	both
8jnk_3P	A28--43 (16)	A80--89 (10)	both
6o39_0P	A27--45 (19)	A90--100 (11)	both
6ye3_2P	A56--66 (11)	A79--103 (25)	both
4zfg_0P	A155--175 (21)	A192--203 (12)	both
8pww_0P	A50--61 (12)	A123--137 (15)	both
8trt_1P	A73--94 (22)	A105--117 (13)	both
8uky_1P	A16--34 (19)	A113--125 (13)	both
6oan_0P	A40--57 (18)	A132--146 (15)	both
5kw9_0P	A118--134 (17)	A152--170 (19)	both
7zxk_1P	A3--19 (17)	A108--130 (23)	both
8t58_0P	A31--60 (30)	A95--120 (26)	both

3 Preprocessing: tiling antigens into assay constructs

The no-antibody (DP4) work starts before any design is generated, by turning an antigen into a set of short peptide constructs to scaffold. Two things drive how we do this.

First, RFD3 needs an input crystal structure. The contig tells RFD3 which residues to hold fixed while it builds a scaffold around them, so those residues have to exist in a PDB. We therefore tile the *PDB-resolved* sequence (the residues actually present in the structure), not the deposited FASTA. 6M0J is the clearest case: its FASTA starts 14 residues before the crystal (chain E begins at residue 333, which is position 15 in the FASTA), so we drop those 14 and tile the 223-residue resolved region. 1D2K (392) and 4WAT (402) match their full chains.

Second, the constructs have to match the linear-epitope assay. Each tile is placed at the C-terminus of a constant 73-residue construct: a glycine–serine filler followed by the ENLYFQGA protease site. Because the prefix is constant, every member is synthesized at a constant length (103 residues for a 30-mer), and the protease site cleaves it down to the bare tile at the end, which is what the assay reads.

3.1 General tiling utility

The tiling step is a single utility (`episcaf_pipeline/tile_fasta.py`) so we can point it at any target. It takes a sequence source (a FASTA, or the `antigen_seq` column of a ledger such as `dp2.parquet`), a window length, and a step, and writes the assay library in the DP2 annotated format (`library_member`, `sequence`, `category`, `model`, `designedSequence`, `designedSequenceLength`, `design_ID`, `target`). Windows are placed at the chosen step; when the last window stops short of the C-terminus, we add one final window so the C-terminal residues are still covered. We pinned the utility to what was already run: re-tiling 1D2K, 4WAT, and 6M0J at a 12-residue window and step 2 reproduces the existing 12-mer library exactly (191, 196, and 107 tiles, every tile and full construct identical).

3.2 Tiled 12-mers, and tiled 30-mers as linear controls

The existing set tiles three antigens into 12-mers. The next category is tiled 30-mers, which serve as the *linear controls*: unscaffolded constructs (model `(none)`) that present the bare peptide, exactly what the scaffolded designs are meant to beat. For those we tile all 59 mAb-target antigens (sequences taken directly from `dp2.parquet`) plus the three tiled antigens, at a 30-residue window and step 6.

3.3 Why the design count is not (tiles $\times$ top-k)

A natural expectation is that taking the top k designs per epitope gives k times the number of tiles. It does not, and the gap is informative. Each tile is one epitope that RFD3 scaffolds into many designs (seeds $\times$ ProteinMPNN sequences), which are then AlphaFold3-validated and gated; we keep at most k per epitope, so the selected total is $\sum_{\text{epitopes}} \min(k, n_{\text{gated}})$. In practice the per-epitope yield is not the bottleneck: every epitope that survives has well over k designs (median $\sim$190 scored). The shortfall is whole epitopes that drop out, because a tile whose residues are unresolved in the crystal cannot be aligned to the native antigen and is discarded at scoring (`crystal_epi_incomplete`). Table 2 shows this for top-5 selection.

antigen	tiles	epitopes lost (crystal)	epitopes kept	top-5 designs
1D2K	191	0	191	951
6M0J	107	15	92	450
4WAT	196	40	156	780

Table 2: Selected designs track (epitopes kept) $\times 5$, not (tiles) $\times 5$. 1D2K loses no epitopes; 6M0J and 4WAT lose 15 and 40 tiles to unresolved crystal regions. The small residuals (951 vs 955, 450 vs 460) are epitopes with fewer than 5 designs clearing the 2.5 Å gate.

4 RFD1 versus RFD3 on the same epitopes

With the metrics reproduced, we compared the two backbone generators head to head: the same DP3 epitopes run through RFdiffusion1 (Lawson's set) and RFdiffusion3, each followed by ProteinMPNN and AlphaFold3, and scored against the identical four filters. The RFD1 data here is Lawson's deposited set (`dp2.parquet`), which is missing AlphaFold3 results for 26 870 of its 151 232 designs; that is a property of his dataset rather than of our pipeline, so we compare over designs that actually have an AF3 result. On that basis the two are comparable, with RFD1 modestly higher: RFD1+MPNN passes 840/124 362 (0.68%) and RFD3+MPNN 751/150 936 (0.50%). RFD3 is therefore at least an even trade for RFD1 on this task, which is the basis for standardizing on RFD3 going forward. The per-metric distributions are comparable (Fig. 1), though RFD3 puts a bit more mass at low epitope and overall RMSD.

We are really comparing two sequence distributions. Each protocol samples sequences from a backbone-conditioned distribution $p(s \mid b)$ and is judged by the same pass rate (Eq. 1), so a difference in $\hat{P}_{\text{pass}}$ comes from a difference in those distributions over comparable backbones. ProteinMPNN optimizes $p(s \mid b)$ directly through its inverse-folding objective, while RFD3 produces it only as a by-product of all-atom denoising. The two landing this close is the result worth keeping.

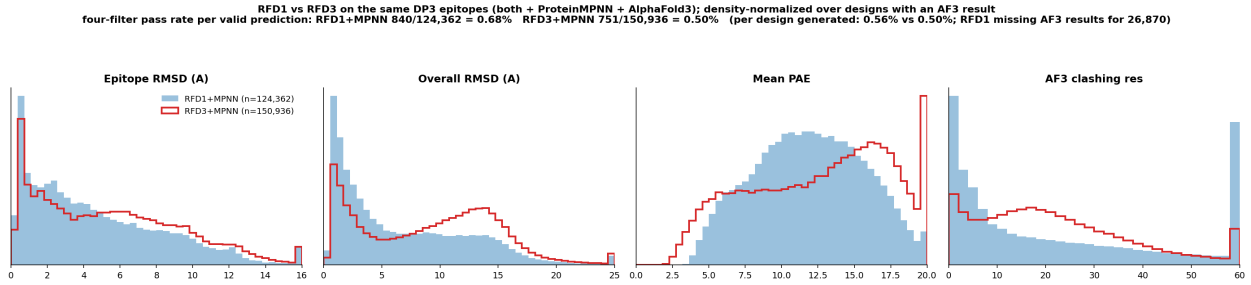

Figure 1: RFD1+MPNN (blue, filled) versus RFD3+MPNN (red, outline) on the same DP3 epitopes, across epitope RMSD, overall RMSD, mean PAE, and AF3 clashing residues, density-normalized over designs with an AF3 result (per-trial n in the legend). Pass rates and the missing-prediction asymmetry are noted in the title. Regenerated by `episcaf_analysis/viz/plot_rfd1_vs_rfd3.py`.

The aggregate pass counts hide a more interesting difference in *where* the passes land (Fig. 2). RFD1's passes are highly concentrated: a single target (`7ox3_0P`) accounts for 630 of its 840 passes, and just two targets account for the large majority. RFD3 spreads its 751 passes across more targets (16 with at least one pass, versus 10 for RFD1), and its per-target leaders are different (`4xwo_5P`, `8pww_0P`, and `8db4_0P` all pass well under RFD3 but barely or not at all under RFD1). For a

library meant to cover many epitopes, that broader coverage is more useful than a similar total piled onto one target.

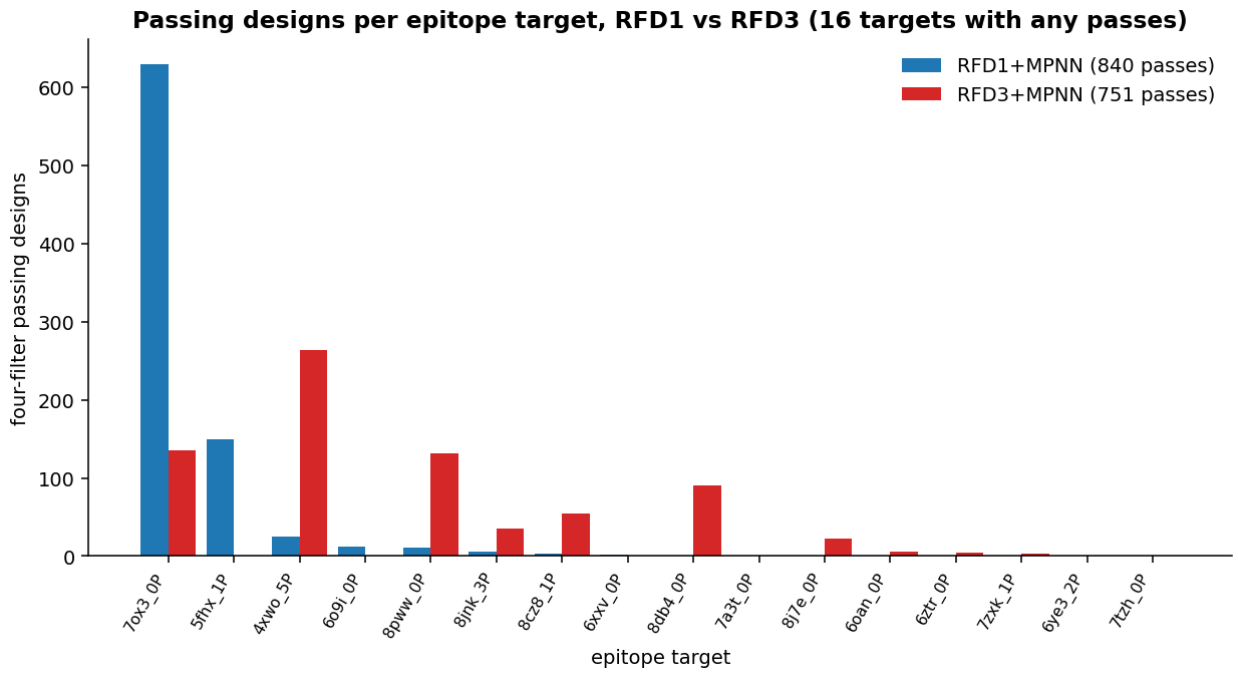

Figure 2: Four-filter passing designs per epitope target, RFD1+MPNN (blue) versus RFD3+MPNN (red). RFD1 is dominated by `7ox3_0P`; RFD3 distributes passes across more targets. Regenerated by `episcaf_analysis/viz/plot_passes_per_epitope.py`.

5 Rethinking the filters: global metrics bias toward rigid folds

With the pipeline reproduced and trusted (Section 2), the first thing we changed was the filtering. The original filters score a design with two global continuous metrics, overall RMSD and mean PAE, that treat the epitope and the scaffold as one unit. We inherited this from standard de novo practice, but it biases selection toward globally rigid, well-ordered folds and quietly rejects designs whose scaffold is flexible while the epitope is well positioned (or the reverse). Splitting each metric into epitope- and scaffold-specific terms (Section 2.4) is what lets us see, and recover, those designs.

To compare design sets on a common footing we use epitope-chunk RMSD < 2.5 Å as a single bar. All three tiled 12-mer antigens pass it more often than the pooled DP3 set:

set	pass rate (< 2.5 Å)	median epitope RMSD (Å)
DP3 / mAb (pooled)	30.0%	5.00
6M0J	47.0%	2.65
1D2K	60.2%	1.85
4WAT	84.4%	0.43

Even the hardest 12-mer antigen (6M0J) clears the bar more often than DP3. Figure 3 shows the full six-metric comparison.

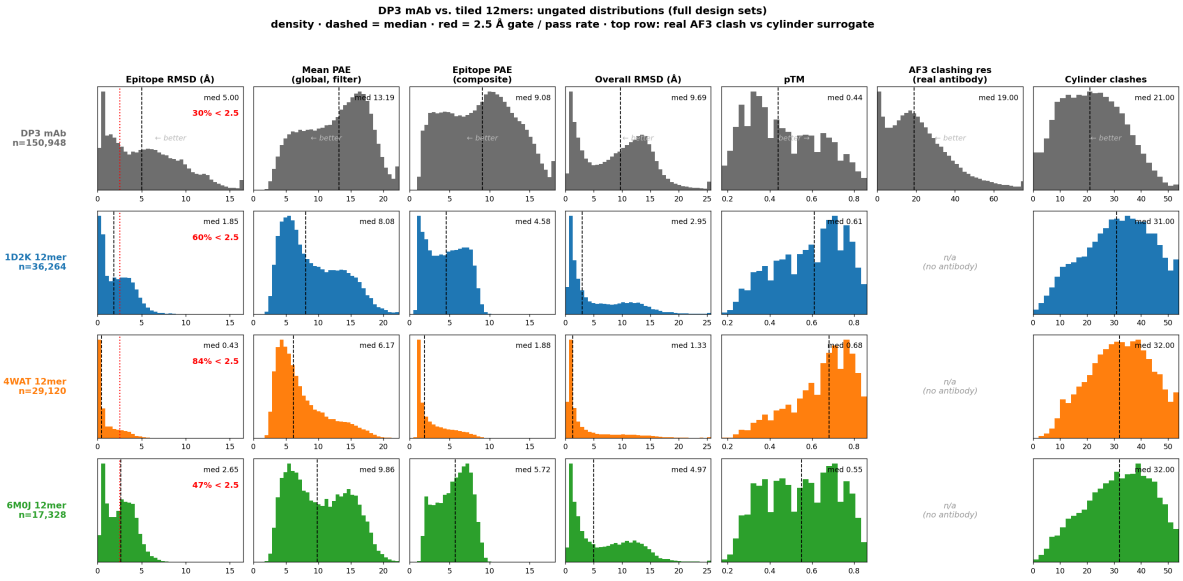

Figure 3: Ungated metric distributions (full design sets), DP3 mAb versus the three 12-mer antigens, across epitope RMSD, global mean PAE (the four-filter metric), epitope PAE (the decomposed metric the composite scores on), overall RMSD, pTM, AF3 clashing residues (antibody set only), and cylinder clashes. Dashed line marks the median; red marks the 2.5 Å epitope-RMSD gate with the per-set pass rate. Regenerated by `notebooks/dp3_vs_12mer_distributions.ipynb`.

Caveats (important for interpretation). This comparison is not yet apples-to-apples. The 12-mer epitope is a single contiguous 12-residue stretch, while DP3 epitopes are larger and often multi-island, so 12-mer RMSD and PAE are mechanically lower in part by definition, and the pooled DP3 30% averages over heterogeneous epitopes. The clean control is to stratify within DP3 by island count and size (Section 9). The cylinder counts are also not comparable across sets as absolute numbers. The 12-mer medians (∼31–32) run higher than DP3 (∼21) mainly because the designs are a fixed length (∼104 residues) and the 12-mer epitope is small, which leaves ∼91 non-epitope scaffold residues whose Cα atoms can land in a fixed cylinder against fewer for DP3's larger epitopes. That is a scaffold-mass effect, not more real clashing, which is why we apply the native-aware carve and rank the cylinder per antigen instead of against one absolute cutoff. As a check, the DP3 cylinder median (≈ 21) tracks its real `af3_n_clash_res` median (≈ 19).

Global versus epitope PAE. The two PAE columns show why we decompose, and they are easy to confuse. The four-filter (and Lawson's original) uses the *global* mean PAE (< 5); on this DP3 set it sits high, a median of 13.2 with only ∼7% of designs below 5. The scaffold region drags that value up, since AlphaFold3 predicts it less confidently, while the epitope itself is much better resolved (epitope PAE median 9.1, ∼23% below 5). So a design can hold its epitope well and still fail the global PAE filter on scaffold uncertainty that need not matter for binding. That is the bias the decomposed metrics relax, and it is why we show both columns and score the epitope-specific PAE rather than the global one.

6 Composite scoring and per-epitope selection

Decomposing the metrics gives more knobs, which raised the next question: how to combine them into one ranking and take the best designs per epitope. We do this with the config-driven composite scorer (Section 2). It gates on epitope-chunk RMSD, transforms each metric to a higher-is-better score within its scope (per antigen for the 12-mer set, pooled for DP3), weight-sums, and keeps the top designs per epitope. Over the full 12-mer set the gate keeps 54 539 of 82 712 designs ($\approx 65.9\%$ pooled) before ranking `[regen]`, consistent with the per-antigen rates above.

The scorer is deliberately simple: one metric per input, each with its own activation, weighted and summed, which makes it a single-layer perceptron with hand-set weights. It is still in flux. The per-metric transform is currently a percentile within the population, so a design's score depends on the cohort it sits in. Whether to move to a population-independent sigmoid (an absolute, saturating per-metric score) is an open choice we carry in Section 9.

7 Scaffolding without an antibody: the cylinder surrogate

Most targets have no solved antibody, so the four-filter ground truth does not apply: there is no antibody to clash against. We extend the workflow to these antigens by tiling them into 12-residue epitopes and replacing the antibody-clash filter with a native-antigen-aware cylinder surrogate (Section 2): a cylinder fitted to the epitope plane along the antibody-approach normal that counts non-epitope scaffold mass intruding into the volume an approaching antibody would need (Fig. 4).

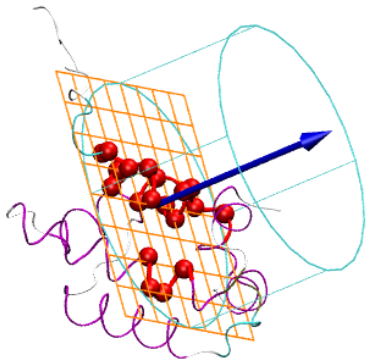

Figure 4: The accessibility cylinder. Epitope residues (red) sit on the scaffold (magenta); the epitope Cα atoms define a plane (orange grid) whose outward normal (blue arrow, the antibody-approach direction) orients the cylinder (cyan). Scaffold Cα atoms inside the cylinder count against accessibility. Rendered in VMD by `known_antigen/analysis/cylinder_vis/visualize_cylinder.tcl`.

A naive cylinder over-penalizes, because some scaffold mass sits exactly where the native antigen already sits, a volume a real antibody clearly tolerates since it binds the native antigen there. The

native-aware carve fixes this: among truly clash-free designs, we drop scaffold Cα atoms within 1 Å of a native-antigen heavy atom. Of the 3186 DP3 designs with zero true antibody clash, the number the plain cylinder flags as heavily penalized (count ≥ 10) falls from 1112 to 384 once the native footprint is carved out, so 728 **false-positive penalties are removed**, while genuinely clashing designs are left essentially untouched (Figs. 5 and 6).

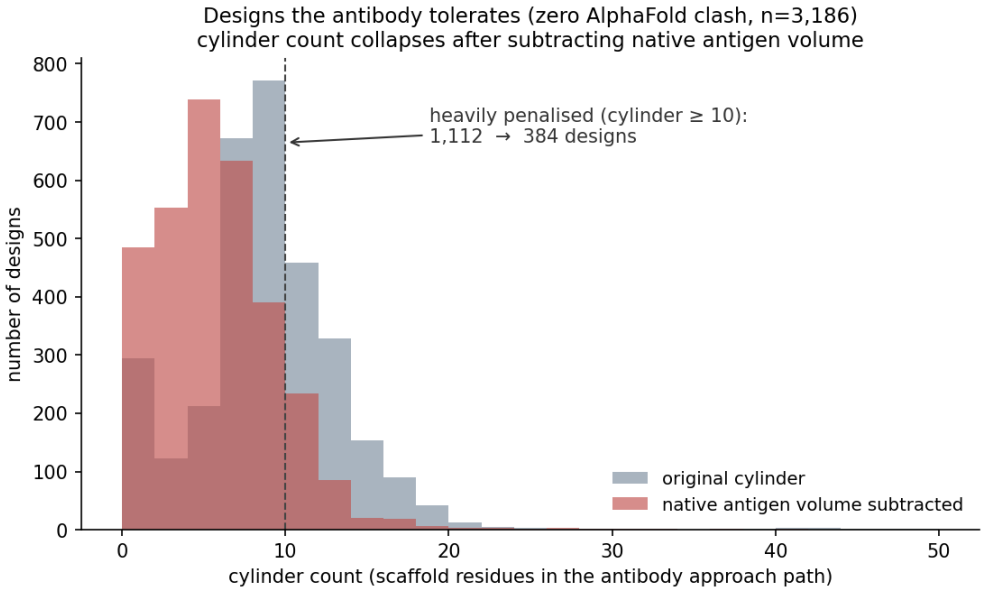

Figure 5: Cylinder count among clash-free DP3 designs (zero AlphaFold3 antibody clash, $n = 3186$), before (grey) and after (red) subtracting the native-antigen volume. The heavily penalized tail (count $\geq$ 10) collapses from 1112 to 384. Regenerated by `episcaf_analysis/viz/plot_fp_reduction.py --native metrics_native_cyl.csv --mode dist`.

A pipeline constraint worth recording: crystal gaps. We measure 12-mer epitope fidelity by aligning the AlphaFold3 epitope onto the native crystal epitope, matched by residue number (`build_12mer_metrics.py`). When a tile's epitope residues are unresolved in the crystal (a gap in the deposited structure), the native Cα anchors are missing, the Kabsch alignment cannot be formed, and the tile is dropped with status `crystal_epi_incomplete`. So which 12-mer epitopes are scorable at all depends on crystal coverage: an antigen with gaps over the tiled region contributes fewer (or no) scorable tiles, regardless of design quality. This is exactly what sets the epitopes-kept counts in Table 2 (6M0J and 4WAT lose 15 and 40 tiles).

7.1 Interpreting the cylinder: where passing designs land

The cylinder is easiest to read by overlaying, for each metric, the full design population against the designs we would actually advance (Fig. 7). For DP3 the advanced set is the four-filter passes (751 designs); for DP4 it is the top three per epitope by composite score. On every geometry and confidence metric the passing designs concentrate where we expect: low RMSD, low PAE, high pTM. The cylinder column is the interesting one. The passing DP3 designs, which we know the antibody accepts, sit at low cylinder counts (median near 10), while the DP4 top-3 sit higher (medians near 25–30). This is the scaffold-mass effect again, not a sign the DP4 designs are worse:

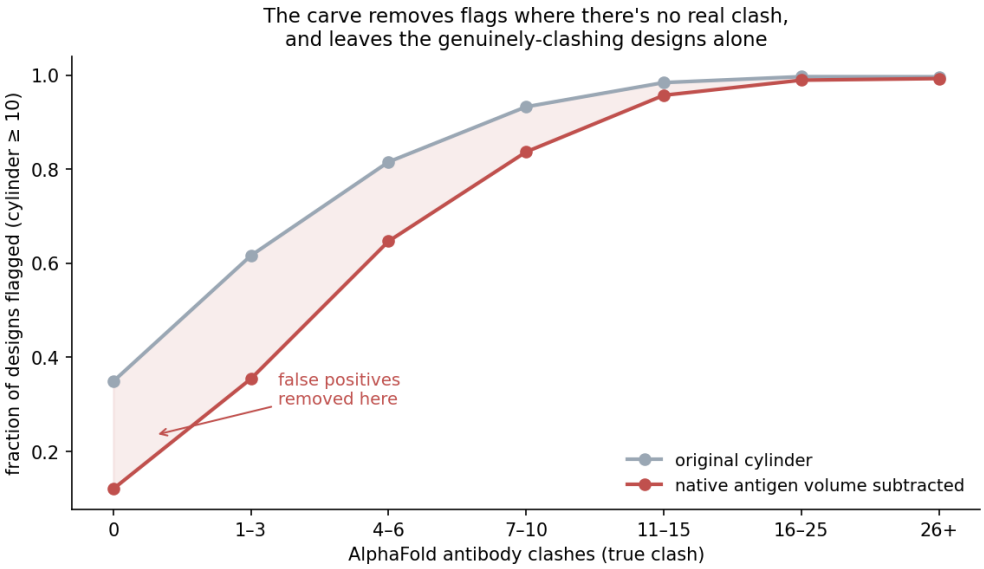

Figure 6: The carve removes flags selectively. Fraction of designs flagged (cylinder ≥ 10) across bins of true antibody clash, before versus after the carve. The gap is large where there is no real clash (false positives removed) and closes where the clash is real (true signal retained). `--mode selectivity`.

with a 12-residue epitope on a ∼104-residue design, more non-epitope scaffold mass falls in the fixed cylinder even among the designs we would pick. This is why we lean on the native-aware carve and per-antigen scoring rather than one absolute cutoff, and it gives us a concrete reference for a tolerable cylinder count, namely the DP3 passing range.

The cylinder is a good stand-in for the real filter. On DP3, where the true antibody clash is known, it predicts clash-free designs (`af3_n_clash_res`=0) with AUC 0.93 (`cylinder_native_aware` 0.935, a little better than the plain count 0.924; clash-free versus clashing medians of 5 versus 22). So it stands in well for the filter we lose on DP4, and the native-aware carve helps the ranking, not just the picture (`episcaf_analysis/cylinder_surrogate_auc.py`).

8 DP3 difficulty by epitope island count

A first cut at the island-stratification control (Section 9): we split the DP3 design set (RFD1+MPNN, `dp2.parquet`) by epitope island count, where an island is a spatially contiguous patch of epitope residues and the count is the number of fixed segments in the contig (the `epitope_chunks` field). Of the 59 epitopes, 13 are single-island and 46 are two-island; none have more under this grouping. Restricting to designs with an AF3 result, that is 549 single-island and 123 813 two-island designs.

Across every four-filter metric the single-island designs sit at better values (Fig. 8): median epitope-chunk RMSD 2.22 vs 3.86 Å, median global PAE 9.5 vs 11.9, and fewer AF3 clashes. The binary pass rate runs the other way (single-island 0.36% vs two-island 0.68%), but that is misleading: the two-island rate is carried almost entirely by one easy outlier (`7ox3_0P`), while the single-island set has no comparable outlier and is broadly easier. So on the distributions, fewer islands is easier to scaffold, consistent with the multi-island difficulty argument in Section 1; the binary rate just hides it. What remains for the clean control is to split further by island *size*, since the single-island epitopes are also shorter on average.

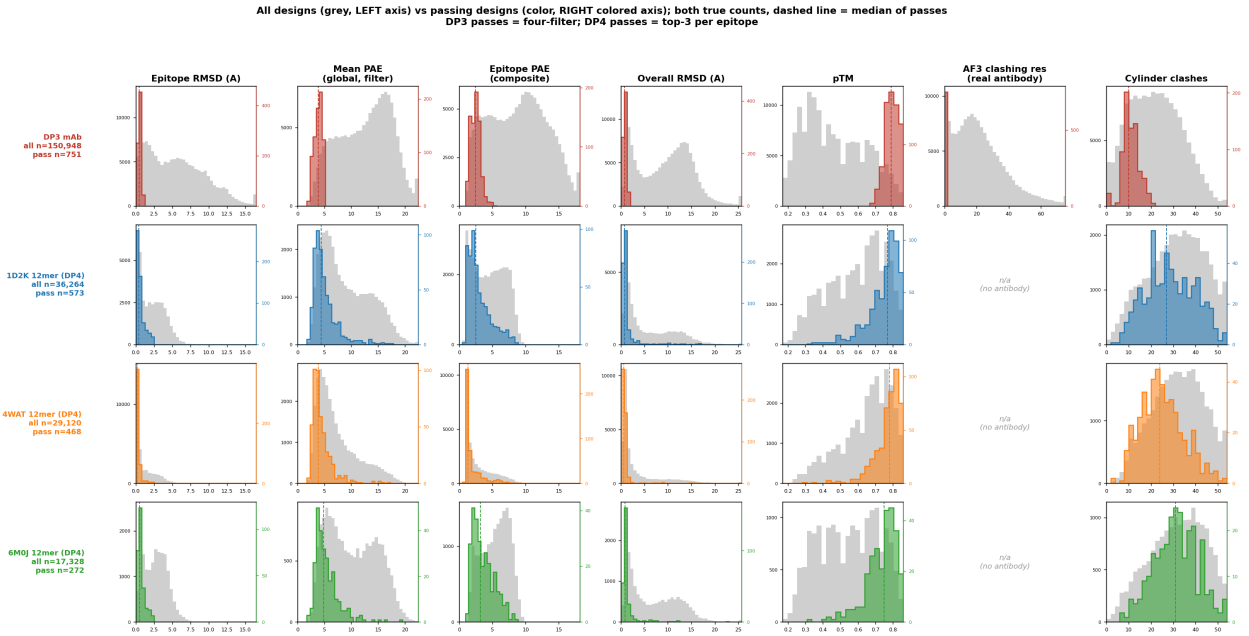

Figure 7: All designs (grey, left axis) and passing designs (color, right colored axis), both in true counts, per metric and per row; the dashed line marks the median of the passes. The twin axes keep the full population and the much smaller passing set each at its own scale, so the shape of the accepted distribution and where it lands are both visible. DP3 passes are the four-filter set; DP4 passes are the top 3 per epitope by composite score. Regenerated by `episcaf_analysis/viz/plot_passes_overlay.py`.

9 Open questions and next steps

1. **Within-DP3 island stratification (the clean control).** We have split by island count (Section 8): single-island designs are distributionally easier on every metric. What remains is to split by island *size*, since the single-island epitopes are also shorter on average, to separate the "contiguous-12 is easier" confound from a real scaffolding-difficulty effect. The per-epitope island metadata is in `dp2.parquet` (`epitope_chunks`, `contig_string`).

2. **Move to a sigmoid transform (planned).** The composite currently maps each metric to a percentile within its population, which is non-differentiable and population-dependent, so it cannot be fit and a score does not transfer from one set to another. A sigmoid (fixed midpoint and slope k per metric) gives an absolute, saturating, differentiable score, which both lets the weights be learned and lets a DP3-trained score transfer to DP4. The scorer already supports it (`transform=sigmoid` in `presets.py`); midpoints can be seeded from the DP3 separations (e.g. a cylinder midpoint near 15, between the clash-free and clashing medians). This is the prerequisite for fitting; the percentile version stays the default until then.

3. **Defer fitting until we have an experimental label.** The composite is a single-layer perceptron with hand-set weights (feed-forward, no backprop), and the natural next step is to learn the weights. We are deliberately *not* fitting yet: the only label available now is the in-silico `is_pass`, which is defined by the four filters themselves, so fitting to it would largely re-derive the filters. The plan instead is to generate designs at scale, prioritize them with the current composite, run the top picks through the PepSeq assay, and fit the weights against

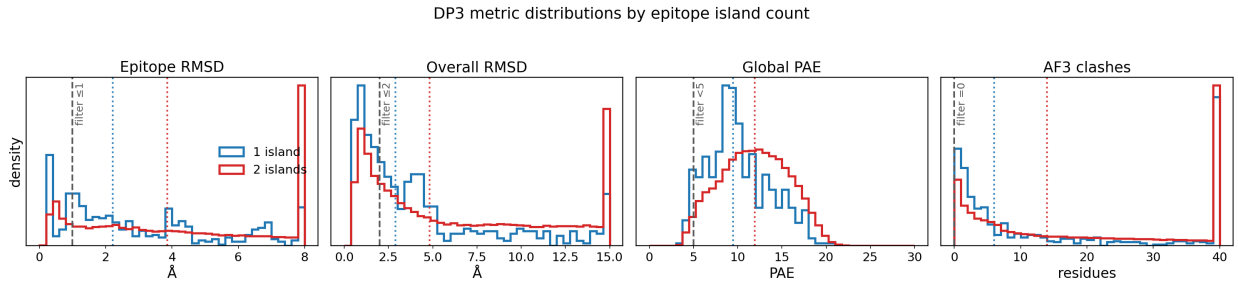

Figure 8: DP3 design metrics (RFD1+MPNN, `dp2.parquet`) split by epitope island count: blue = single-island (n = 549), red = two-island (n = 123 813); only designs with an AF3 result are included. Panels, left to right: epitope-chunk RMSD, overall RMSD, global mean PAE, and number of AF3 clashing residues, the four selection metrics. Distributions are area-normalized (density) because the groups differ in size by ∼200×. Dotted vertical lines mark each group's median; dashed grey lines mark the four-filter thresholds (epitope RMSD $\leq$ 1, overall RMSD $\leq$ 2, PAE $<$ 5, clashes = 0). Single-island designs are shifted toward better values on every metric (median epitope RMSD 2.22 vs 3.86 Å, median PAE 9.5 vs 11.9). Regenerated by `episcaf_analysis/viz/plot_island_split.py`.

experimental binding, the non-circular target. The cylinder weight in particular is already justified by data: on DP3 it predicts the true antibody clash at AUC 0.93 (Section 7), so it earns real weight even before a global fit.

4. **Confirm the canonical DP3 metrics builder.** Determine whether `compute_metrics_mpnn.py` (currently archived) is the live RFD3+MPNN metrics producer, and promote it into `episcaf_analysis/` if so (see `docs/MIGRATION.md`).

5. **Validate the DP3 scoring path** end-to-end with `--preset antibody`, confirming `cylinder_ca_clashes` is the correct accessibility column there.

6. **Finish the 12-mer AlphaFold3 data migration** and verify landing counts against the manifest.

7. **Pin down the crystal-gap dropouts.** We can already count them locally: 6M0J and 4WAT lose 15 and 40 tiles to unresolved crystal regions (Table 2). What is left is to confirm these against the cluster pre-filter status (the explicit `crystal_epi_incomplete` entries) and to check whether the lost tiles fall in contiguous runs, i.e. whether whole regions go un-scorable from gaps.

References